

Recuerdos

Amaru está comprando recuerdos (souvenirs) en una tienda extranjera. Hay N **tipos** de recuerdos. En la tienda hay infinitos recuerdos de cada tipo.

Cada tipo de recuerdo tiene un precio fijo. Esto es, un recuerdo del tipo i ($0 \leq i < N$) tiene un precio de $P[i]$ monedas, donde $P[i]$ es un entero positivo.

Amaru sabe que los tipos de recuerdos están ordenados de manera decreciente por precio, y que los precios de los recuerdos son distintos. En concreto, $P[0] > P[1] > \dots > P[N-1] > 0$. Además, ha podido conocer el valor de $P[0]$. Por desgracia, Amaru no dispone de más información sobre los precios de los recuerdos.

Para comprar algunos recuerdos, Amaru realizará una serie de transacciones con el vendedor.

Cada transacción consta de los siguientes pasos:

1. Amaru entrega un número (positivo) de monedas al vendedor.
2. El vendedor pone estas monedas en una pila sobre la mesa en el cuarto trasero de la tienda, donde Amaru no puede verlas.
3. El vendedor considera cada tipo de recuerdo $0, 1, \dots, N-1$ en ese orden, uno por uno. Cada tipo es considerado **exactamente una vez** por transacción.
 - Al considerar el tipo de recuerdo i , si el número actual de monedas en la pila es al menos $P[i]$, entonces
 - el vendedor retira $P[i]$ monedas de la pila, y
 - el vendedor pone un recuerdo de tipo i sobre la mesa.
4. El vendedor entrega a Amaru todas las monedas que quedan en la pila y todos los recuerdos de la mesa.

Nótese que no hay monedas ni recuerdos sobre la mesa antes de que comience la transacción.

Tu tarea consiste en pedir a Amaru que realice un cierto número de transacciones, de modo que:

- en cada transacción compre **al menos un** recuerdo, y
- en total compre **exactamente** i recuerdos del tipo i , para cada i tal que $0 \leq i < N$. Nótese que esto significa que Amaru no debe comprar ningún recuerdo del tipo 0.

Amaru no necesita minimizar el número de transacciones y tiene un suministro ilimitado de monedas.

Detalles de implementación

Debes implementar la siguiente función.

```
void buy_souvenirs(int N, long long P0)
```

- N : el número de tipos de recuerdos.
- $P0$: el valor de $P[0]$.
- Esta función se llama exactamente una vez para cada caso de prueba.

La función anterior puede hacer llamadas a la siguiente función para pedir a Amaru que realice una transacción:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : el número de monedas entregadas por Amaru al vendedor.
- La función devuelve un par. El primer elemento del par es un arreglo L que contiene los tipos de recuerdos que se han comprado (en orden creciente). El segundo elemento es un entero R , que corresponde al número de monedas devueltas a Amaru tras la transacción.
- Se requiere que $P[0] > M \geq P[N - 1]$. La condición $P[0] > M$ asegura que Amaru no compra ningún recuerdo del tipo 0, y $M \geq P[N - 1]$ asegura que Amaru compra al menos un recuerdo. Si no se cumplen estas condiciones, tu solución recibirá el veredicto `Output isn't correct: Invalid argument`. Nótese que, a diferencia de $P[0]$, el valor de $P[N - 1]$ no se proporciona en la entrada.
- La función puede llamarse como máximo 5000 veces en cada caso de prueba.

El comportamiento del evaluador es **no adaptativo**. Esto significa que la secuencia de precios P se fija antes de llamar a `buy_souvenirs`.

Restricciones

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i tal que $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N - 1$
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$ $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$
6	33	Sin restricciones adicionales

Ejemplo

Considera la siguiente llamada.

```
buy_souvenirs(3, 4)
```

Existen $N = 3$ tipos de recuerdos y $P[0] = 4$. Observa que sólo hay tres secuencias posibles de precios P : $[4, 3, 2]$, $[4, 3, 1]$, y $[4, 2, 1]$.

Digamos que `buy_souvenirs` llama a `transaction(2)`. Supongamos que la llamada retorna $([2], 1)$, lo que significa que Amaru compró un recuerdo de tipo 2 y el vendedor le devolvió 1 moneda. Observa que esto nos permite deducir que $P = [4, 3, 1]$, ya que:

- Para $P = [4, 3, 2]$, `transaction(2)` habría retornado $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` habría retornado $([1], 0)$.

Entonces `buy_souvenirs` puede llamar a `transaction(3)`, que retorna $([1], 0)$, lo que significa que Amaru compró un recuerdo del tipo 1 y el vendedor le devolvió 0 monedas. Hasta ahora, en total, él ha comprado un recuerdo de tipo 1 y un recuerdo de tipo 2.

Por último, `buy_souvenirs` puede llamar a `transaction(1)`, que retorna $([2], 0)$, lo que significa que Amaru compró un recuerdo del tipo 2. Nótese que aquí también podríamos haber utilizado `transaction(2)`. En este momento, en total Amaru tiene un recuerdo de tipo 1 y dos recuerdos de tipo 2, como se requiere.

Evaluador de ejemplo

Formato de entrada:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Formato de salida:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Aquí $Q[i]$ es el número de recuerdos del tipo i comprados en total para cada i tal que $0 \leq i < N$.